ADVANCED DATA SCIENCE

Days 53–60 Complete Learning Guide

Experiment Tracking • Model Governance • CI/CD • Explainable AI • Responsible AI • Capstone

Final phase of the 60-Day Data Science Learning Journey

How to use this guide

This guide consolidates the final eight days into one practical reference. Study each concept, run the accompanying Python capstone, inspect the generated files, and extend the project. The project uses synthetic records, not real customer data.

Learning outcomes

• Track experiments and compare model runs using consistent metrics.

• Manage model versions, metadata, reproducibility, and rollback.

• Understand CI/CD checks for machine learning projects.

• Explain model behavior with feature importance, SHAP, and LIME.

• Review fairness, privacy, transparency, and model limitations.

• Build an end-to-end classification pipeline and basic monitoring report.

• Present your work clearly in a technical portfolio.

Environment setup

```
python -m venv .venv
pip install pandas numpy scikit-learn joblib
# Optional experiment tracking
pip install mlflow
python Days-53-60-End-to-End-Churn-Analytics-Capstone.py
```

Day 53. MLflow and experiment tracking

Experiment tracking creates a record of each model run. A run can contain parameters, metrics, artifacts, and model files. This makes comparisons repeatable and easier to review.

What to log

• Parameters: algorithm, tree count, depth, learning rate, and random seed.

• Metrics: precision, recall, F1, ROC-AUC, MAE, or RMSE, depending on the task.

• Artifacts: reports, charts, feature lists, and model files.

• Context: dataset version, code version, timestamp, and environment.

```
import mlflow
mlflow.set_experiment("customer-churn")
with mlflow.start_run():
    mlflow.log_param("n_estimators", 200)
    mlflow.log_metric("f1_score", 0.82)  # illustrative only
# Start local interface with: mlflow ui
```

Compare experiments using the same split and evaluation method. Do not select a model from one metric alone.

Practice

Train models with 50, 100, and 200 trees. Log each run and compare F1 and ROC-AUC. Explain the trade-off behind your choice.

Day 54. Model registry and version management

A model artifact needs supporting information. Record its version, training data reference, feature schema, parameters, metrics, dependencies, and intended use.

Versioning principles

• Use meaningful versions such as 1.0.0 and 1.1.0.

• Preserve the last approved model for rollback.

• Keep preprocessing and estimator together in one pipeline.

• Record the exact environment and feature schema.

• Validate compatibility before loading a model in another environment.

```
import joblib, json
joblib.dump(pipeline, "model_v1.joblib")
metadata = {"version":"1.0.0",
            "algorithm":"RandomForestClassifier",
            "features":["tenure_months","monthly_charges"]}
with open("model_v1_metadata.json","w") as f:
    json.dump(metadata,f,indent=2)
loaded = joblib.load("model_v1.joblib")
```

Only load trusted Joblib or pickle files. These formats can execute code during loading.

Practice

Save two versions with separate metadata. Compare their test metrics and define a rollback rule.

Day 55. CI/CD for machine learning

Continuous Integration automatically checks code changes. Continuous Delivery or Deployment prepares or releases a validated application. ML projects also need data and model quality checks.

Basic GitHub Actions example

```
name: Python checks
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install pytest scikit-learn pandas numpy
      - run: pytest
```

Useful checks

• Syntax, unit tests, and dependency checks.

• Input schema and missing-value validation.

• Model performance against a baseline.

• Threshold checks for important metrics.

• Artifact integrity and reproducibility.

• Manual approval before high-impact releases.

A passing workflow means the configured checks passed. It does not prove that a model is fair, safe, or useful.

Practice

Write a test that sends one valid record through your pipeline and checks that exactly one prediction returns.

Day 56. Explainable AI

Explainable AI methods help people understand model behavior. An explanation does not prove that a prediction is correct or that a feature causes an outcome.

Common methods

• Built-in feature importance gives a quick global ranking for supported models.

• Permutation importance measures the score change after shuffling a feature.

• SHAP estimates feature contributions under a defined explanation framework.

• LIME fits a simpler local approximation around one prediction.

• Partial Dependence Plots show average prediction changes as a feature varies.

```
import shap
explainer = shap.TreeExplainer(tree_model)
values = explainer.shap_values(X_sample)
shap.summary_plot(values, X_sample)
```

SHAP return shapes and APIs can vary by model and version. Check the installed version documentation. Distinguish global explanations from explanations for one record.

Practice

Choose one test record. Explain its prediction with local feature contributions, then compare that explanation with global feature importance.

Day 57. Responsible AI and fairness

Responsible AI includes fairness, privacy, transparency, accountability, reliability, security, and human oversight. Strong overall accuracy can hide poor performance for a subgroup.

Review questions

• Was the data collected for a legitimate purpose?

• Are some groups missing or underrepresented?

• Do false positives or false negatives affect groups differently?

• Can a person challenge or review a high-impact decision?

• Is the model being used for the purpose it was evaluated for?

• Can sensitive data be minimized and protected?

Fairness metrics

Demographic parity compares positive prediction rates across groups. Equal opportunity compares true-positive rates. These metrics represent different fairness goals and may conflict. Choose measures based on the application and potential harms.

The capstone includes a descriptive age-group review using synthetic records. It is not a complete fairness audit or a legal compliance assessment.

Practice

Compare group-level prediction rates, recall, and false-positive rates. Check sample sizes and explain what the results do and do not establish.

Day 58. End-to-end machine learning

An end-to-end workflow connects validation, preprocessing, training, evaluation, saving, inference, and monitoring. Scikit-learn Pipeline and ColumnTransformer keep transformations attached to the estimator.

Why use a pipeline

• Apply the same transformations during training and inference.

• Reduce preprocessing mismatch and data leakage risk.

• Save preprocessing and the estimator as one artifact.

• Make cross-validation and deployment more consistent.

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
```

```
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.ensemble import RandomForestClassifier
# Build numeric and categorical transformers,
# combine them with ColumnTransformer,
# then place preprocessing and classifier in Pipeline.
```

Use a stratified split when appropriate. Keep the test set untouched until final evaluation. Select metrics based on the cost of errors.

Practice

Run the supplied script. Inspect metrics, confusion matrix, feature importance, subgroup review, saved model, and prediction CSV.

Day 59. Portfolio and career preparation

A strong portfolio explains the problem, data, method, evaluation, limitations, and reproduction steps. Show your reasoning, not only a final score.

Suggested project structure

```
customer-churn-capstone/
  data/
  notebooks/
  src/
  tests/
  outputs/
  README.md
  requirements.txt
  .gitignore
```

README checklist

• Problem and intended user.

• Dataset source or clear synthetic-data statement.

• Installation and run instructions.

• Feature definitions and preprocessing.

• Baseline and final metrics.

• Selected visuals and error analysis.

• Limitations, risks, and ethical considerations.

• Future improvements and references.

Interview questions

• What is data leakage and how do you prevent it?

• Why can accuracy mislead on imbalanced data?

• Explain precision, recall, F1, and ROC-AUC.

• Why use a preprocessing pipeline?

• What is the difference between validation and test data?

• How do you compare model versions?

• What is model drift?

• How would you explain one prediction?

• What is the difference between CI and CD?

• How would you monitor a model after deployment?

• What are fairness trade-offs?

• How would you roll back a poor release?

A 60-day plan builds foundations and momentum. It does not guarantee employment. Continue with statistics, SQL, deep learning, feature engineering, cloud fundamentals, and communication.

Day 60. Final capstone: Customer analytics and churn prediction

The capstone generates synthetic customer records, preprocesses numeric and categorical columns, trains a Random Forest classifier, evaluates predictions, exports feature importance, reviews subgroup statistics, saves model metadata, reloads the model, and demonstrates a basic input mean-shift report.

Architecture

```
Synthetic records
       |
Train/test split
       |
Preprocessing pipeline
(imputation, scaling, one-hot encoding)
       |
Random Forest classifier
       |
       +-- Evaluation metrics and confusion matrix
       +-- Feature importance
       +-- Subgroup review
       +-- Saved model and metadata
       +-- Prediction export
       +-- Basic input drift report
       +-- Optional MLflow tracking
```

Outputs created by the Python project

• synthetic_customer_data.csv: generated dataset.

• churn_pipeline.joblib: preprocessing plus trained model.

• model_metadata.json: version, model, and metrics.

• test_predictions.csv: actual labels, predictions, and probabilities.

• feature_importance.csv: ranked transformed-feature importance.

• subgroup_review.csv: descriptive group-level statistics.

• basic_drift_report.csv: simple numeric mean comparison.

Limitations

• Synthetic data cannot establish real-world performance.

• Mean comparison is a teaching example, not a robust drift detector.

• Subgroup review is not a full fairness audit.

• Feature importance does not establish causation.

• The script does not deploy an API or automate retraining.

• Production use needs governance, security, validation, monitoring, and human review.

Extension milestones

• Add unit tests for input schema, predictions, and metric calculations.

• Compare Logistic Regression, Random Forest, and Gradient Boosting.

• Use cross-validation and log runs in MLflow.

• Add SHAP explanations for selected predictions.

• Build a Streamlit dashboard.

• Document model approval and rollback.

• Add CI checks for every pull request.

• Consider deployment only after validation and access controls.

Suggested capstone rubric

Area	Points	Evidence
Data quality	15	Schema, missing values, leakage prevention
Modeling	20	Baseline, validation, model selection
Evaluation	15	Relevant metrics and error analysis
Explainability	10	Global and local explanations
Responsible AI	10	Limitations and subgroup review
Reproducibility	15	Pipeline, metadata, experiment log
Communication	15	README, visuals, clear conclusions

References and continued learning

• Scikit-learn User Guide: https://scikit-learn.org/stable/user_guide.html

• MLflow Documentation: https://mlflow.org/docs/latest/

• SHAP Documentation: https://shap.readthedocs.io/

• LIME project: https://github.com/marcotcr/lime

• GitHub Actions: https://docs.github.com/actions

• NIST AI Risk Management Framework: https://www.nist.gov/itl/ai-risk-management-framework

Continue by building, testing, documenting, and reviewing real projects. Be precise about what your evidence supports.